

Auto-Complete, Explain Code & Suggest Alternatives

Modern AI-based programming tools such as **GitHub Copilot, TabNine, and Amazon CodeWhisperer** assist developers by automatically completing code, explaining existing code, and suggesting better or optimized alternatives. These features make coding faster, easier, and more efficient.

1. Auto-Complete

Meaning

Auto-complete is a feature where the system **predicts and suggests code** while the programmer is typing. It helps reduce typing effort and avoids syntax errors.

How It Works

The AI tool analyzes:

- The programming language
- The current file

Auto-generating APIs and boilerplate code

Auto-Generating APIs and Boilerplate Code

AI-powered development tools such as **GitHub Copilot, Amazon CodeWhisperer, and ChatGPT** can automatically generate **APIs and boilerplate code**, allowing developers to focus more on business logic rather than repetitive setup tasks.

1. What is Boilerplate Code?

Definition

Boilerplate code is **repetitive and standard code** that is required to start an application but does not contain core business logic.

Examples of Boilerplate Code

- Server setup
- Database connection
- API routing structure
- Configuration files
- Error handling structure

Example (Express Server Boilerplate)

```
const express = require('express');  
  
const app = express();
```

```
app.use(express.json());
```

```
app.listen(3000, () => {  
  console.log("Server running on port 3000");  
});
```

2. What Does Auto-Generating APIs Mean?

Auto-generating APIs means using **AI tools or frameworks** to create:

- API endpoints
- Request and response structures
- Controllers and routes
- Database models

This reduces manual coding and development time.

3. Auto-Generating REST APIs

Example Prompt

“Create a REST API for managing student data using Node.js and Express”

AI-Generated API (Example)

```
app.get('/students', (req, res) => {  
  res.json({ message: "List of students" });  
});
```

```
app.post('/students', (req, res) => {  
  res.json({ message: "Student added" });  
});
```

4. Auto-Generating CRUD APIs

CRUD stands for:

- **Create**
- **Read**

- **Update**
- **Delete**

Example: CRUD API for Students

```
app.put('/students/:id', (req, res) => {
  res.send("Student updated");
});
```

```
app.delete('/students/:id', (req, res) => {
  res.send("Student deleted");
});
```

5. Auto-Generating APIs with Database Integration

Example: MongoDB + Mongoose API

```
const mongoose = require('mongoose');
```

```
const StudentSchema = new mongoose.Schema({
  name: String,
  age: Number
});
```

```
module.exports = mongoose.model('Student', StudentSchema);
```

AI tools can generate:

- Schema
- Controller
- Routes
- Database connection

6. Benefits of Auto-Generating APIs and Boilerplate Code

Benefit	Description
Faster development	Saves time by avoiding repetitive coding

Benefit	Description
Consistency	Maintains standard structure
Fewer errors	Reduces syntax and logical mistakes
Beginner-friendly	Helps students learn API structure

7. Real-Time Example

Scenario

A student needs to build a **college management system**.

With AI Tools

- Generates server setup
- Creates student, course, and faculty APIs
- Connects MongoDB automatically
- Adds basic error handling

Result

Project is completed **faster and more professionally**.

8. Tools Used for Auto-Generation

- **GitHub Copilot** – Code suggestions inside IDE
 - **ChatGPT** – Generates APIs based on prompts
 - **Spring Initializr** – Generates Spring Boot boilerplate
 - **Swagger/OpenAPI** – Auto-generates API structure
-

9. Limitations

- Generated code may need optimization
 - Security issues must be reviewed
 - Over-dependence can affect learning if basics are ignored
-

10. Conclusion

Auto-generating APIs and boilerplate code using AI tools helps developers and students **build applications quickly, efficiently, and correctly**, while allowing them to focus on problem-solving and business logic.

